

ECEN 743: Reinforcement Learning

Advanced Actor-Critic Algorithms

Dileep Kalathil
Texas A&M University

Soft Actor Critic (SAC) Algorithm

References

- (SAC Paper) Haarnoja, et al. “Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor”, *International Conference on Machine Learning (ICML)*, 2018.
- (Updated SAC and explanation) Haarnoja, et al. “Soft actor-critic algorithms and applications”, 2018.
- (Soft Q-learning Paper) Haarnoja, et al. “Reinforcement learning with deep energy-based policies”, *International Conference on Machine Learning (ICML)*, 2017.
- (Convergence analysis of entropy regularized NPG) Cen et al. “Fast global convergence of natural policy gradient methods with entropy regularization”, *Operations Research*, 2022.

Entropy Regularized RL

- Standard RL objective: $\max_{\pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$ where $a_t \sim \pi(s_t, \cdot)$

Entropy Regularized RL

- Standard RL objective: $\max_{\pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$ where $a_t \sim \pi(s_t, \cdot)$
- Entropy regularized RL objective

$$\max_{\pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) - \alpha \log \pi(s_t, a_t))], \text{ where, } a_t \sim \pi(s_t, \cdot)$$

Entropy Regularized RL

- Standard RL objective: $\max_{\pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$ where $a_t \sim \pi(s_t, \cdot)$
- Entropy regularized RL objective

$$\max_{\pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) - \alpha \log \pi(s_t, a_t))], \text{ where, } a_t \sim \pi(s_t, \cdot)$$

- Entropy of a distribution p : $H(p) = -\sum_x p(x) \log p(x)$

Entropy Regularized RL

- Standard RL objective: $\max_{\pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t)]$ where $a_t \sim \pi(s_t, \cdot)$
- Entropy regularized RL objective

$$\max_{\pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) - \alpha \log \pi(s_t, a_t))], \text{ where, } a_t \sim \pi(s_t, \cdot)$$

- Entropy of a distribution p : $H(p) = -\sum_x p(x) \log p(x)$
- So, this is also called maximum entropy RL:

$$\max_{\pi} \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) + \alpha H(\pi(s_t, \cdot)))], \text{ where, } a_t \sim \pi(s_t, \cdot)$$

Why max-entropy?

- Maximum entropy formulation provides a significant improvement in exploration and robustness
- It gives nicer objective functions from an optimization perspective

Soft Values

- Standard value function $V_{\pi, \mu} = \mathbb{E}_{s \sim \mu}[V_{\pi}(s)]$

Soft Values

- Standard value function $V_{\pi,\mu} = \mathbb{E}_{s \sim \mu}[V_{\pi}(s)]$
- Soft value function

$$V_{\pi,\mu}^s = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) - \alpha \log(\pi(s_t, \cdot)))\right], \quad \text{where, } a_t \sim \pi(s_t, \cdot), s_0 \sim \mu$$

Soft Values

- Standard value function $V_{\pi,\mu} = \mathbb{E}_{s \sim \mu}[V_{\pi}(s)]$

- Soft value function

$$V_{\pi,\mu}^s = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) - \alpha \log(\pi(s_t, \cdot)))\right], \text{ where, } a_t \sim \pi(s_t, \cdot), s_0 \sim \mu$$

- Denoting $H(\pi, \mu) = \mathbb{E}_{s \sim \rho_{\pi,\mu}}[H(\pi(s, \cdot))]$, we get

$$V_{\pi,\mu}^s = V_{\pi,\mu} + \alpha H(\pi, \mu)$$

Soft Values

- Standard value function $V_{\pi,\mu} = \mathbb{E}_{s \sim \mu}[V_{\pi}(s)]$

- Soft value function

$$V_{\pi,\mu}^s = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) - \alpha \log(\pi(s_t, \cdot)))\right], \text{ where, } a_t \sim \pi(s_t, \cdot), s_0 \sim \mu$$

- Denoting $H(\pi, \mu) = \mathbb{E}_{s \sim \rho_{\pi,\mu}}[H(\pi(s, \cdot))]$, we get

$$V_{\pi,\mu}^s = V_{\pi,\mu} + \alpha H(\pi, \mu)$$

- Denoting π^* and π^{*s} as the optimal policies of standard and regularized objectives, we get

$$V_{\pi^*,\mu} \leq V_{\pi^{*s},\mu}^s \leq V_{\pi^*,\mu} + \frac{\alpha \log |\mathcal{A}|}{(1 - \gamma)}$$

Soft Values

- Standard value function $V_{\pi,\mu} = \mathbb{E}_{s \sim \mu}[V_{\pi}(s)]$

- Soft value function

$$V_{\pi,\mu}^s = \mathbb{E}\left[\sum_{t=0}^{\infty} \gamma^t (r(s_t, a_t) - \alpha \log(\pi(s_t, \cdot)))\right], \text{ where, } a_t \sim \pi(s_t, \cdot), s_0 \sim \mu$$

- Denoting $H(\pi, \mu) = \mathbb{E}_{s \sim \rho_{\pi,\mu}}[H(\pi(s, \cdot))]$, we get

$$V_{\pi,\mu}^s = V_{\pi,\mu} + \alpha H(\pi, \mu)$$

- Denoting π^* and π^{*s} as the optimal policies of standard and regularized objectives, we get

$$V_{\pi^*,\mu} \leq V_{\pi^{*s},\mu}^s \leq V_{\pi^*,\mu} + \frac{\alpha \log |\mathcal{A}|}{(1 - \gamma)}$$

- So, π^{*s} could also be nearly optimal in terms of the standard value function, as long as the regularization parameter α is chosen to be sufficiently small

Form of the Optimal Soft Policy

- We want to solve:

$$\max_{\pi} \mathbb{E}_{a \sim \pi(s, \cdot)} [Q(s, a) - \alpha \log \pi(s, a)]$$

Form of the Optimal Soft Policy

- We want to solve:

$$\max_{\pi} \mathbb{E}_{a \sim \pi(s, \cdot)} [Q(s, a) - \alpha \log \pi(s, a)]$$

- Note that state s fixed. Writing it as $\sum_a \pi(s, a) Q(s, a) - \alpha \sum_a \pi(s, a) \log \pi(s, a)$, and taking gradient w.r.t. π and equating it zero, we get:

Form of the Optimal Soft Policy

- We want to solve:

$$\max_{\pi} \mathbb{E}_{a \sim \pi(s, \cdot)} [Q(s, a) - \alpha \log \pi(s, a)]$$

- Note that state s fixed. Writing it as $\sum_a \pi(s, a) Q(s, a) - \alpha \sum_a \pi(s, a) \log \pi(s, a)$, and taking gradient w.r.t. π and equating it zero, we get:
- Optimal soft policy has the **softmax** form:

$$\pi^{*s}(s, a) \propto \exp(Q(s, a)/\alpha)$$

Form of the Optimal Soft Policy

- We want to solve:

$$\max_{\pi} \mathbb{E}_{a \sim \pi(s, \cdot)} [Q(s, a) - \alpha \log \pi(s, a)]$$

- Note that state s fixed. Writing it as $\sum_a \pi(s, a) Q(s, a) - \alpha \sum_a \pi(s, a) \log \pi(s, a)$, and taking gradient w.r.t. π and equating it zero, we get:
- Optimal soft policy has the **softmax** form:

$$\pi^{*s}(s, a) \propto \exp(Q(s, a)/\alpha)$$

- More precisely,

$$\pi^{*s}(s, a) = \exp\left(\frac{1}{\alpha}(Q^{*s}(s, a) - V^{*s}(s))\right), \text{ and,}$$

$$V^{*s}(s) = \alpha \log\left(\sum_a \exp(Q^{*s}(s, a)/\alpha)\right)$$

Soft Policy Evaluation

- Define the soft policy evaluation operator T_{π}^s as

$$T_{\pi}^s Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(s, a)} [V(s')], \text{ where, } V(s) = \mathbb{E}_{a \sim \pi(s, \cdot)} [Q(s, a) - \alpha \log \pi(s, a)]$$

Soft Policy Evaluation

- Define the soft policy evaluation operator T_π^s as

$$T_\pi^s Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(s, a)} [V(s')], \text{ where, } V(s) = \mathbb{E}_{a \sim \pi(s, \cdot)} [Q(s, a) - \alpha \log \pi(s, a)]$$

Lemma

- T_π^s is a contraction mapping
- The iteration $Q_{k+1} = T_\pi^s Q_k$ will converge to the **soft Q-value function** Q_π^s , which satisfy the consistency equation

$$Q_\pi^s(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(s, a)} [V_\pi^s(s')], \text{ where, } V_\pi^s(s) = \mathbb{E}_{a \sim \pi(s, \cdot)} [Q_\pi^s(s, a) - \alpha \log \pi(s, a)]$$

Soft Bellman Operator

- Consider the natural extension of the standard Bellman operator

$$T^s Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(s, a)} \left[\max_{\pi(s', \cdot)} \mathbb{E}_{a' \sim \pi(s', \cdot)} [Q(s', a') - \alpha \log \pi(s', a')] \right]$$

Soft Bellman Operator

- Consider the natural extension of the standard Bellman operator

$$T^s Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(s, a)} \left[\max_{\pi(s', \cdot)} \mathbb{E}_{a' \sim \pi(s', \cdot)} [Q(s', a') - \alpha \log \pi(s', a')] \right]$$

- From the previous slide, this is equivalent to

$$T^s Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(s, a)} \left[\alpha \log \left(\sum_{a'} \exp(Q(s', a') / \alpha) \right) \right]$$

Soft Bellman Operator

- Consider the natural extension of the standard Bellman operator

$$T^s Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(s, a)} \left[\max_{\pi(s', \cdot)} \mathbb{E}_{a' \sim \pi(s', \cdot)} [Q(s', a') - \alpha \log \pi(s', a')] \right]$$

- From the previous slide, this is equivalent to

$$T^s Q(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim P(s, a)} \left[\alpha \log \left(\sum_{a'} \exp(Q(s', a') / \alpha) \right) \right]$$

Proposition

- T^s is a contraction w.r.t. $\|\cdot\|_\infty$ with a contraction coefficient γ
- Q^{*s} is the unique fixed point of T^s

Soft Policy Iteration

- Similar to the standard policy iteration. Start with an initial policy π_0 . For each k , perform soft policy evaluation and soft policy update

Soft Policy Iteration

- Similar to the standard policy iteration. Start with an initial policy π_0 . For each k , perform soft policy evaluation and soft policy update
- **Soft policy evaluation:** Given the policy π_k , compute $Q_{\pi_k}^s$

Soft Policy Iteration

- Similar to the standard policy iteration. Start with an initial policy π_0 . For each k , perform soft policy evaluation and soft policy update
- **Soft policy evaluation:** Given the policy π_k , compute $Q_{\pi_k}^s$
- **Soft policy update:** Given $Q_{\pi_k}^s$, update the policy to get $\pi_{k+1}(s, a) = \frac{1}{Z_{\pi_k}(s)} \exp(Q_{\pi_k}^s(s, a)/\alpha)$

Soft Policy Iteration

- Similar to the standard policy iteration. Start with an initial policy π_0 . For each k , perform soft policy evaluation and soft policy update
- **Soft policy evaluation:** Given the policy π_k , compute $Q_{\pi_k}^s$
- **Soft policy update:** Given $Q_{\pi_k}^s$, update the policy to get $\pi_{k+1}(s, a) = \frac{1}{Z_{\pi_k}(s)} \exp(Q_{\pi_k}^s(s, a)/\alpha)$

Proposition

- 1 *Monotone improvement:*

$$V_{\pi_{k+1}}^s(s) - V_{\pi_k}^s(s) = \mathbb{E}_{s' \sim \rho_{\pi_{k+1}, \delta(s)}} \left[\frac{\alpha}{(1 - \gamma)} D_{\text{KL}}(\pi_{k+1}(s, \cdot), \pi_k(s, \cdot)) \right]$$

Soft Policy Iteration

- Similar to the standard policy iteration. Start with an initial policy π_0 . For each k , perform soft policy evaluation and soft policy update
- **Soft policy evaluation:** Given the policy π_k , compute $Q_{\pi_k}^s$
- **Soft policy update:** Given $Q_{\pi_k}^s$, update the policy to get $\pi_{k+1}(s, a) = \frac{1}{Z_{\pi_k}(s)} \exp(Q_{\pi_k}^s(s, a)/\alpha)$

Proposition

- ❶ *Monotone improvement:*

$$V_{\pi_{k+1}}^s(s) - V_{\pi_k}^s(s) = \mathbb{E}_{s' \sim \rho_{\pi_{k+1}, \delta(s)}} \left[\frac{\alpha}{(1 - \gamma)} D_{\text{KL}}(\pi_{k+1}(s, \cdot), \pi_k(s, \cdot)) \right]$$

- ❷ *Convergence rate:*

$$\begin{aligned} \|Q^{*s} - Q_{\pi_k}^s\|_{\infty} &\leq \gamma^k \|Q^{*s} - Q_{\pi_0}^s\|_{\infty} \\ \|\log \pi^{*s} - \log \pi_{k+1}\|_{\infty} &\leq \frac{2}{\alpha} \gamma^k \|Q^{*s} - Q_{\pi_0}^s\|_{\infty} \end{aligned}$$

Off-policy Learning

- Policy optimization algorithms such as PG, TRPO, PPO are on-policy learning algorithms

Off-policy Learning

- Policy optimization algorithms such as PG, TRPO, PPO are on-policy learning algorithms
- On-policy algorithms require new samples to be collected for every update of the policy; Data collected from previous iterates is discarded in on-policy algorithms

Off-policy Learning

- Policy optimization algorithms such as PG, TRPO, PPO are on-policy learning algorithms
- On-policy algorithms require new samples to be collected for every update of the policy; Data collected from previous iterates is discarded in on-policy algorithms
- On-policy data collection is expensive (in terms of the number of gradient steps and samples per step needed); this is exacerbated in high dimensional complex tasks

Off-policy Learning

- Policy optimization algorithms such as PG, TRPO, PPO are on-policy learning algorithms
- On-policy algorithms require new samples to be collected for every update of the policy; Data collected from previous iterates is discarded in on-policy algorithms
- On-policy data collection is expensive (in terms of the number of gradient steps and samples per step needed); this is exacerbated in high dimensional complex tasks
- Off-policy learning algorithms can reuse the data from all learning episodes (recall the replay buffer in DQN)

Off-policy Learning

- Policy optimization algorithms such as PG, TRPO, PPO are on-policy learning algorithms
- On-policy algorithms require new samples to be collected for every update of the policy; Data collected from previous iterates is discarded in on-policy algorithms
- On-policy data collection is expensive (in terms of the number of gradient steps and samples per step needed); this is exacerbated in high dimensional complex tasks
- Off-policy learning algorithms can reuse the data from all learning episodes (recall the replay buffer in DQN)
- SAC is an off-policy policy optimization algorithm

Soft Actor-Critic Algorithm: Q-Value update

- Function approximation for soft Q-function Q_θ and policy π_ϕ

Soft Actor-Critic Algorithm: Q-Value update

- Function approximation for soft Q-function Q_θ and policy π_ϕ
- Instead of running evaluation for each policy π_k and performing improvement using $Q_{\pi_k}^s$, SAC alternate between optimizing θ, ϕ networks with stochastic gradient descent

Soft Actor-Critic Algorithm: Q-Value update

- Function approximation for soft Q-function Q_θ and policy π_ϕ
- Instead of running evaluation for each policy π_k and performing improvement using $Q_{\pi_k}^s$, SAC alternate between optimizing θ, ϕ networks with stochastic gradient descent
- **Soft Q-function parameter update:** Recall that $V_\pi^s(s) = \mathbb{E}_{a \sim \pi(s, \cdot)}[Q_\pi^s(s, a) - \alpha \log \pi(s, a)]$. So, we define the loss function $L_Q(\theta)$ as

$$L_Q(\theta) = (1/2) \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} [(Q_\theta(s, a) - (r + \gamma \mathbb{E}_{a' \sim \pi_\phi(s, \cdot)}[Q_{\bar{\theta}}(s', a') - \alpha \log \pi(s', a')]))^2],$$

where, the target $\bar{\theta}$ is an exponentially moving average of past soft Q-function weights.

Soft Actor-Critic Algorithm: Q-Value update

- Function approximation for soft Q-function Q_θ and policy π_ϕ
- Instead of running evaluation for each policy π_k and performing improvement using $Q_{\pi_k}^s$, SAC alternate between optimizing θ, ϕ networks with stochastic gradient descent
- **Soft Q-function parameter update:** Recall that $V_\pi^s(s) = \mathbb{E}_{a \sim \pi(s, \cdot)}[Q_\pi^s(s, a) - \alpha \log \pi(s, a)]$. So, we define the loss function $L_Q(\theta)$ as

$$L_Q(\theta) = (1/2) \mathbb{E}_{(s, a, r, s') \sim \mathcal{D}} [(Q_\theta(s, a) - (r + \gamma \mathbb{E}_{a' \sim \pi_\phi(s, \cdot)}[Q_{\bar{\theta}}(s', a') - \alpha \log \pi(s', a')]))^2],$$

where, the target $\bar{\theta}$ is an exponentially moving average of past soft Q-function weights.

- The gradient of $L_Q(\theta)$ estimated using a single sample (s, a, r, s') and $a' \sim \pi_\phi(s', \cdot)$ as

$$\hat{\nabla}_\theta L_Q(\theta) = \nabla_\theta Q_\theta(s, a) (Q_\theta(s, a) - (r + \gamma \mathbb{E}_{a' \sim \pi_\phi(s, \cdot)}[Q_{\bar{\theta}}(s', a') - \alpha \log \pi(s', a')]))$$

Soft Actor-Critic Algorithm: Policy update

- **Policy parameter update:** SAC updates policy using the loss function

$$L_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \left[D_{\text{KL}}(\pi_{\phi}(s, \cdot), \frac{1}{Z_{\theta}(s)} \exp(Q_{\theta}(s, \cdot)/\alpha)) \right]$$

Soft Actor-Critic Algorithm: Policy update

- **Policy parameter update:** SAC updates policy using the loss function

$$L_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \left[D_{\text{KL}}(\pi_{\phi}(s, \cdot), \frac{1}{Z_{\theta}(s)} \exp(Q_{\theta}(s, \cdot)/\alpha)) \right]$$

- This can be rewritten as

$$L_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \mathbb{E}_{a \sim \pi_{\phi}(s, \cdot)} [\alpha \log \pi_{\phi}(s, a) - Q_{\theta}(s, a)]$$

Soft Actor-Critic Algorithm: Policy update

- **Policy parameter update:** SAC updates policy using the loss function

$$L_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \left[D_{\text{KL}}(\pi_{\phi}(s, \cdot), \frac{1}{Z_{\theta}(s)} \exp(Q_{\theta}(s, \cdot)/\alpha)) \right]$$

- This can be rewritten as

$$L_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \mathbb{E}_{a \sim \pi_{\phi}(s, \cdot)} [\alpha \log \pi_{\phi}(s, a) - Q_{\theta}(s, a)]$$

- Use $a = f_{\phi}(\epsilon; s)$, where ϵ is an input noise vector sampled from a fixed distribution such as Gaussian

Soft Actor-Critic Algorithm: Policy update

- **Policy parameter update:** SAC updates policy using the loss function

$$L_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \left[D_{\text{KL}}(\pi_{\phi}(s, \cdot), \frac{1}{Z_{\theta}(s)} \exp(Q_{\theta}(s, \cdot)/\alpha)) \right]$$

- This can be rewritten as

$$L_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \mathbb{E}_{a \sim \pi_{\phi}(s, \cdot)} [\alpha \log \pi_{\phi}(s, a) - Q_{\theta}(s, a)]$$

- Use $a = f_{\phi}(\epsilon; s)$, where ϵ is an input noise vector sampled from a fixed distribution such as Gaussian
- Then, we can rewrite the above loss function as

$$L_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \mathbb{E}_{\epsilon \sim \mathcal{N}} [\log \pi_{\phi}(s, f_{\phi}(\epsilon; s)) - Q_{\theta}(s, f_{\phi}(\epsilon; s))]$$

Soft Actor-Critic Algorithm: Policy update

- **Policy parameter update:** SAC updates policy using the loss function

$$L_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \left[D_{\text{KL}}(\pi_{\phi}(s, \cdot), \frac{1}{Z_{\theta}(s)} \exp(Q_{\theta}(s, \cdot)/\alpha)) \right]$$

- This can be rewritten as

$$L_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \mathbb{E}_{a \sim \pi_{\phi}(s, \cdot)} [\alpha \log \pi_{\phi}(s, a) - Q_{\theta}(s, a)]$$

- Use $a = f_{\phi}(\epsilon; s)$, where ϵ is an input noise vector sampled from a fixed distribution such as Gaussian
- Then, we can rewrite the above loss function as

$$L_{\pi}(\phi) = \mathbb{E}_{s \sim \mathcal{D}} \mathbb{E}_{\epsilon \sim \mathcal{N}} [\log \pi_{\phi}(s, f_{\phi}(\epsilon; s)) - Q_{\theta}(s, f_{\phi}(\epsilon; s))]$$

- The gradient of $L_{\pi}(\phi)$ estimated using a single sample $s \sim \mathcal{D}$ and $\epsilon \sim \mathcal{N}$ as

$$\hat{\nabla}_{\phi} L_{\pi}(\phi) = \nabla_{\phi} \log \pi_{\phi}(s, a) + (\nabla_a \log \pi_{\phi}(s, a) - \nabla_a Q_{\theta}(s, a)) \nabla_{\phi} f_{\phi}(s, a)|_{a=f_{\phi}(\epsilon; s)}$$

Soft Actor-Critic Algorithm: Double Q-Network

- Recall the double DQN algorithm!

Soft Actor-Critic Algorithm: Double Q-Network

- Recall the double DQN algorithm!
- **Over estimation** bias in standard Q-learning: Using the same network to select the max action and estimate its value will lead to overestimating the Q-value

Soft Actor-Critic Algorithm: Double Q-Network

- Recall the double DQN algorithm!
- **Over estimation** bias in standard Q-learning: Using the same network to select the max action and estimate its value will lead to overestimating the Q-value
- SAC overcome this issue by training two Q-functions, with parameters θ_1 and θ_2

Soft Actor-Critic Algorithm: Double Q-Network

- Recall the double DQN algorithm!
- **Over estimation** bias in standard Q-learning: Using the same network to select the max action and estimate its value will lead to overestimating the Q-value
- SAC overcome this issue by training two Q-functions, with parameters θ_1 and θ_2
- The minimum of the the soft Q-functions is then used for the stochastic gradient updates (this idea was proposed in the TD3 paper)

Soft Actor-Critic Algorithm

Algorithm 1 Soft Actor-Critic

Input: θ_1, θ_2, ϕ ▷ Initial parameters
 $\bar{\theta}_1 \leftarrow \theta_1, \bar{\theta}_2 \leftarrow \theta_2$ ▷ Initialize target network weights
 $\mathcal{D} \leftarrow \emptyset$ ▷ Initialize an empty replay pool
for each iteration **do**
 for each environment step **do**
 $\mathbf{a}_t \sim \pi_\phi(\mathbf{a}_t | \mathbf{s}_t)$ ▷ Sample action from the policy
 $\mathbf{s}_{t+1} \sim p(\mathbf{s}_{t+1} | \mathbf{s}_t, \mathbf{a}_t)$ ▷ Sample transition from the environment
 $\mathcal{D} \leftarrow \mathcal{D} \cup \{(\mathbf{s}_t, \mathbf{a}_t, r(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})\}$ ▷ Store the transition in the replay pool
 end for
 for each gradient step **do**
 $\theta_i \leftarrow \theta_i - \lambda_Q \hat{\nabla}_{\theta_i} J_Q(\theta_i)$ for $i \in \{1, 2\}$ ▷ Update the Q-function parameters
 $\phi \leftarrow \phi - \lambda_\pi \hat{\nabla}_\phi J_\pi(\phi)$ ▷ Update policy weights
 $\alpha \leftarrow \alpha - \lambda \hat{\nabla}_\alpha J(\alpha)$ ▷ Adjust temperature
 $\bar{\theta}_i \leftarrow \tau \theta_i + (1 - \tau) \bar{\theta}_i$ for $i \in \{1, 2\}$ ▷ Update target network weights
 end for
end for
Output: θ_1, θ_2, ϕ ▷ Optimized parameters

SAC Paper

Twin Delayed Deep Deterministic policy gradient (TD3) Algorithm

References

- (DPG Paper) Silver, et al. “Deterministic policy gradient algorithms”, *International Conference on Machine Learning (ICML)*, 2014.
- (DDPG Paper) Lillicrap, et al. “Continuous control with deep reinforcement learning”, *International Conference on Learning Representation (ICLR)*, 2016.
- (TD3 Paper) Fujimoto , et al. “Addressing Function Approximation Error in Actor-Critic Methods”, *International Conference on Machine Learning (ICML)*, 2018.

Deterministic Policy Gradient

- Recall the (stochastic) policy gradient theorem

$$\nabla V_{\pi_{\theta}} = \frac{1}{(1 - \gamma)} \mathbb{E}_{(s,a) \sim \rho_{\pi_{\theta}}(\cdot, \cdot)} [\nabla_{\theta} \log \pi_{\theta}(s, a) Q_{\pi_{\theta}}(s, a)]$$

Deterministic Policy Gradient

- Recall the (stochastic) policy gradient theorem

$$\nabla V_{\pi_{\theta}} = \frac{1}{(1 - \gamma)} \mathbb{E}_{(s,a) \sim \rho_{\pi_{\theta}}(\cdot, \cdot)} [\nabla_{\theta} \log \pi_{\theta}(s, a) Q_{\pi_{\theta}}(s, a)]$$

Theorem (Deterministic Policy Gradient Theorem (Silver et. al., 2014))

Let π_{θ} be a deterministic policy. Then,

$$\nabla_{\theta} V_{\pi_{\theta}} = \frac{1}{(1 - \gamma)} \mathbb{E}_{s \sim \rho_{\pi_{\theta}}(\cdot)} [\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q_{\pi_{\theta}}(s, a)|_{a=\pi_{\theta}(s)}]$$

Deterministic Policy Gradient

- Recall the (stochastic) policy gradient theorem

$$\nabla V_{\pi_{\theta}} = \frac{1}{(1 - \gamma)} \mathbb{E}_{(s,a) \sim \rho_{\pi_{\theta}}(\cdot, \cdot)} [\nabla_{\theta} \log \pi_{\theta}(s, a) Q_{\pi_{\theta}}(s, a)]$$

Theorem (Deterministic Policy Gradient Theorem (Silver et. al., 2014))

Let π_{θ} be a deterministic policy. Then,

$$\nabla_{\theta} V_{\pi_{\theta}} = \frac{1}{(1 - \gamma)} \mathbb{E}_{s \sim \rho_{\pi_{\theta}}(\cdot)} [\nabla_{\theta} \pi_{\theta}(s) \nabla_a Q_{\pi_{\theta}}(s, a)|_{a=\pi_{\theta}(s)}]$$

- Deep Deterministic Policy Gradient (DDPG) algorithm is the deep learning version of DPG algorithm
- DDPG is an off-policy learning algorithm

Q-value Estimation

- In deep Q-learning, the Q-function parameter is typically updated using stochastic gradient descent with respect to the loss function

$$L_Q(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left((r + \gamma \max_{a'} Q_{\theta'}(s', a')) - Q_{\theta}(s, a) \right)^2 \right]$$

Q-value Estimation

- In deep Q-learning, the Q-function parameter is typically updated using stochastic gradient descent with respect to the loss function

$$L_Q(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left((r + \gamma \max_{a'} Q_{\theta'}(s', a')) - Q_{\theta}(s, a) \right)^2 \right]$$

- We are solving a prediction (least squares regression) $\min_{f \in \mathcal{F}} \mathbb{E}_{(x,y)} [(y - f(x))^2]$

Q-value Estimation

- In deep Q-learning, the Q-function parameter is typically updated using stochastic gradient descent with respect to the loss function

$$L_Q(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left((r + \gamma \max_{a'} Q_{\theta'}(s', a')) - Q_{\theta}(s, a) \right)^2 \right]$$

- We are solving a prediction (least squares regression) $\min_{f \in \mathcal{F}} \mathbb{E}_{(x,y)} [(y - f(x))^2]$
 - ▶ The optimal solution is $f(x) = \mathbb{E}[y|x]$

Q-value Estimation

- In deep Q-learning, the Q-function parameter is typically updated using stochastic gradient descent with respect to the loss function

$$L_Q(\theta) = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\left((r + \gamma \max_{a'} Q_{\theta'}(s', a')) - Q_{\theta}(s, a) \right)^2 \right]$$

- We are solving a prediction (least squares regression) $\min_{f \in \mathcal{F}} \mathbb{E}_{(x,y)} [(y - f(x))^2]$
 - ▶ The optimal solution is $f(x) = \mathbb{E}[y|x]$
- So, the optimal solution (with infinite data) is $Q_{\theta}(s, a) = \mathbb{E}_{s' \sim P(\cdot|s,a)} [(r + \gamma \max_{a'} Q_{\theta'}(s', a'))]$

Overestimation Bias in Q-value Estimation

Lemma (Interchanging maximum and expectation)

Consider the function $g : \mathcal{X} \times \mathcal{B} \rightarrow \mathbb{R}$, denoted as $g(x, b)$. Then,

$$\mathbb{E}_{x \sim p_X} [\max_{b \in \mathcal{B}} g(x, b)] \geq \max_{b \in \mathcal{B}} \mathbb{E}_{x \sim p_X} [g(x, b)]$$

Overestimation Bias in Q-value Estimation

Lemma (Interchanging maximum and expectation)

Consider the function $g : \mathcal{X} \times \mathcal{B} \rightarrow \mathbb{R}$, denoted as $g(x, b)$. Then,

$$\mathbb{E}_{x \sim p_X} [\max_{b \in \mathcal{B}} g(x, b)] \geq \max_{b \in \mathcal{B}} \mathbb{E}_{x \sim p_X} [g(x, b)]$$

- Suppose the target Q-function has an additional zero-mean estimation error ϵ , which results in the regression target $y = (r + \gamma \max_{a'} (Q_{\theta'}(s', a') + \epsilon))$

Overestimation Bias in Q-value Estimation

Lemma (Interchanging maximum and expectation)

Consider the function $g : \mathcal{X} \times \mathcal{B} \rightarrow \mathbb{R}$, denoted as $g(x, b)$. Then,

$$\mathbb{E}_{x \sim p_X} [\max_{b \in \mathcal{B}} g(x, b)] \geq \max_{b \in \mathcal{B}} \mathbb{E}_{x \sim p_X} [g(x, b)]$$

- Suppose the target Q-function has an additional zero-mean estimation error ϵ , which results in the regression target $y = (r + \gamma \max_{a'} (Q_{\theta'}(s', a') + \epsilon))$
- Then, the optimal solution (with infinite data) is $\mathbb{E}_{s' \sim P(\cdot|s,a)} \mathbb{E}_{\epsilon} [(r + \gamma \max_{a'} (Q_{\theta'}(s', a') + \epsilon))]$, which will be **larger** than the desired solution $\mathbb{E}_{s' \sim P(\cdot|s,a)} [(r + \gamma \max_{a'} Q_{\theta'}(s', a'))]$

Overestimation Bias in Q-value Estimation

Lemma (Interchanging maximum and expectation)

Consider the function $g : \mathcal{X} \times \mathcal{B} \rightarrow \mathbb{R}$, denoted as $g(x, b)$. Then,

$$\mathbb{E}_{x \sim p_X} [\max_{b \in \mathcal{B}} g(x, b)] \geq \max_{b \in \mathcal{B}} \mathbb{E}_{x \sim p_X} [g(x, b)]$$

- Suppose the target Q-function has an additional zero-mean estimation error ϵ , which results in the regression target $y = (r + \gamma \max_{a'} (Q_{\theta'}(s', a') + \epsilon))$
- Then, the optimal solution (with infinite data) is $\mathbb{E}_{s' \sim P(\cdot|s,a)} \mathbb{E}_{\epsilon} [(r + \gamma \max_{a'} (Q_{\theta'}(s', a') + \epsilon))]$, which will be **larger** than the desired solution $\mathbb{E}_{s' \sim P(\cdot|s,a)} [(r + \gamma \max_{a'} Q_{\theta'}(s', a'))]$
- As a result, even initially zero-mean error can cause value updates to result in a consistent overestimation bias

Overestimation Bias in Q-value Estimation

Lemma (Interchanging maximum and expectation)

Consider the function $g : \mathcal{X} \times \mathcal{B} \rightarrow \mathbb{R}$, denoted as $g(x, b)$. Then,

$$\mathbb{E}_{x \sim p_X} [\max_{b \in \mathcal{B}} g(x, b)] \geq \max_{b \in \mathcal{B}} \mathbb{E}_{x \sim p_X} [g(x, b)]$$

- Suppose the target Q-function has an additional zero-mean estimation error ϵ , which results in the regression target $y = (r + \gamma \max_{a'} (Q_{\theta'}(s', a') + \epsilon))$
- Then, the optimal solution (with infinite data) is $\mathbb{E}_{s' \sim P(\cdot|s,a)} \mathbb{E}_{\epsilon} [(r + \gamma \max_{a'} (Q_{\theta'}(s', a') + \epsilon))]$, which will be **larger** than the desired solution $\mathbb{E}_{s' \sim P(\cdot|s,a)} [(r + \gamma \max_{a'} Q_{\theta'}(s', a'))]$
- As a result, even initially zero-mean error can cause value updates to result in a consistent overestimation bias
- This errors is then propagated through the learning updates accumulating to large values

Overestimation Bias in Q-value Estimation

Lemma (Interchanging maximum and expectation)

Consider the function $g : \mathcal{X} \times \mathcal{B} \rightarrow \mathbb{R}$, denoted as $g(x, b)$. Then,

$$\mathbb{E}_{x \sim p_X} [\max_{b \in \mathcal{B}} g(x, b)] \geq \max_{b \in \mathcal{B}} \mathbb{E}_{x \sim p_X} [g(x, b)]$$

- Suppose the target Q-function has an additional zero-mean estimation error ϵ , which results in the regression target $y = (r + \gamma \max_{a'} (Q_{\theta'}(s', a') + \epsilon))$
- Then, the optimal solution (with infinite data) is $\mathbb{E}_{s' \sim P(\cdot|s,a)} \mathbb{E}_{\epsilon} [(r + \gamma \max_{a'} (Q_{\theta'}(s', a') + \epsilon))]$, which will be **larger** than the desired solution $\mathbb{E}_{s' \sim P(\cdot|s,a)} [(r + \gamma \max_{a'} Q_{\theta'}(s', a'))]$
- As a result, even initially zero-mean error can cause value updates to result in a consistent overestimation bias
- This errors is then propagated through the learning updates accumulating to large values
- This accumulated error can cause arbitrarily bad states to be estimated as high value, resulting in suboptimal policy updates and divergent behavior.

Mitigating Overestimation Bias

- Double DQN is an idea used to mitigate the overestimation bias in DQN

Mitigating Overestimation Bias

- Double DQN is an idea used to mitigate the overestimation bias in DQN
- Double DQN uses one Q-network to select the max-action and another Q-network to evaluate it

$$y = r + \gamma Q_{\theta'}(s', \arg \max_b Q_{\theta}(s', b))$$

Mitigating Overestimation Bias

- Double DQN is an idea used to mitigate the overestimation bias in DQN
- Double DQN uses one Q-network to select the max-action and another Q-network to evaluate it

$$y = r + \gamma Q_{\theta'}(s', \arg \max_b Q_{\theta}(s', b))$$

- Actor-Critic algorithms do not directly takes the maximization over actions

Mitigating Overestimation Bias

- Double DQN is an idea used to mitigate the overestimation bias in DQN
- Double DQN uses one Q-network to select the max-action and another Q-network to evaluate it

$$y = r + \gamma Q_{\theta'}(s', \arg \max_b Q_{\theta}(s', b))$$

- Actor-Critic algorithms do not directly takes the maximization over actions
- However, gradient w.r.t actions can be considered as local maximization

Mitigating Overestimation Bias

- Double DQN is an idea used to mitigate the overestimation bias in DQN
- Double DQN uses one Q-network to select the max-action and another Q-network to evaluate it

$$y = r + \gamma Q_{\theta'}(s', \arg \max_b Q_{\theta}(s', b))$$

- Actor-Critic algorithms do not directly takes the maximization over actions
- However, gradient w.r.t actions can be considered as local maximization
- So, actor-critic algorithms also faces the overestimation bias problem

Mitigating Overestimation Bias

- Double DQN is an idea used to mitigate the overestimation bias in DQN
- Double DQN uses one Q-network to select the max-action and another Q-network to evaluate it

$$y = r + \gamma Q_{\theta'}(s', \arg \max_b Q_{\theta}(s', b))$$

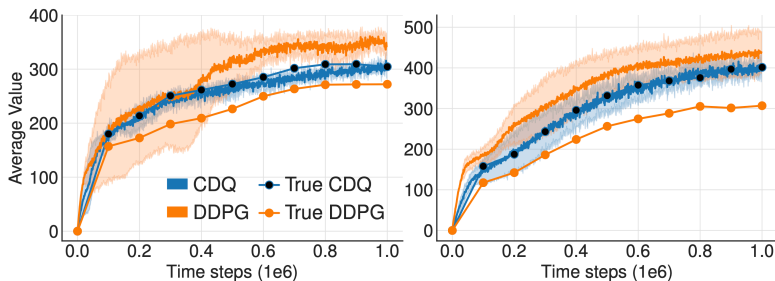
- Actor-Critic algorithms do not directly takes the maximization over actions
- However, gradient w.r.t actions can be considered as local maximization
- So, actor-critic algorithms also faces the overestimation bias problem
- DDPG used a variant of **double Q-learning** to mitigate the overestimation bias

Mitigating Overestimation Bias

- TD3 paper proposed **Clipped Double Q-learning**
 - ▶ Train two (**twin**) Q-networks $Q_{\theta_1}, Q_{\theta_2}$ with the corresponding targets $Q_{\theta'_1}, Q_{\theta'_2}$ networks
 - ▶ Then, select the target for Q-update as $y = r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \pi_\phi(s'))$

Mitigating Overestimation Bias

- TD3 paper proposed **Clipped Double Q-learning**
 - ▶ Train two (**twin**) Q-networks $Q_{\theta_1}, Q_{\theta_2}$ with the corresponding targets $Q_{\theta'_1}, Q_{\theta'_2}$ networks
 - ▶ Then, select the target for Q-update as $y = r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \pi_\phi(s'))$



(a) Hopper-v1

(b) Walker2d-v1

Two More Ideas in TD3

- Delayed policy updates

Two More Ideas in TD3

- Delayed policy updates
 - ▶ Intuitively, critic evaluation has to be completed before performing an actor update

Two More Ideas in TD3

- Delayed policy updates

- ▶ Intuitively, critic evaluation has to be completed before performing an actor update
- ▶ TD3 uses a **delayed** policy update by taking multiple critic update before performing one policy update

Two More Ideas in TD3

- Delayed policy updates
 - ▶ Intuitively, critic evaluation has to be completed before performing an actor update
 - ▶ TD3 uses a **delayed** policy update by taking multiple critic update before performing one policy update
- Target policy smoothing

Two More Ideas in TD3

- Delayed policy updates
 - ▶ Intuitively, critic evaluation has to be completed before performing an actor update
 - ▶ TD3 uses a **delayed** policy update by taking multiple critic update before performing one policy update
- Target policy smoothing
 - ▶ Similar actions should have similar value; need to avoid overfitting to narrow peaks in the value

Two More Ideas in TD3

- Delayed policy updates

- ▶ Intuitively, critic evaluation has to be completed before performing an actor update
- ▶ TD3 uses a **delayed** policy update by taking multiple critic update before performing one policy update

- Target policy smoothing

- ▶ Similar actions should have similar value; need to avoid overfitting to narrow peaks in the value
- ▶ TD3 proposes fitting the value of a small area around the target action by setting

$$y = r + \mathbb{E}_{\epsilon}[Q_{\theta'}(s', \pi_{\phi'}(s') + \epsilon)]$$

Two More Ideas in TD3

- Delayed policy updates

- ▶ Intuitively, critic evaluation has to be completed before performing an actor update
- ▶ TD3 uses a **delayed** policy update by taking multiple critic update before performing one policy update

- Target policy smoothing

- ▶ Similar actions should have similar value; need to avoid overfitting to narrow peaks in the value
- ▶ TD3 proposes fitting the value of a small area around the target action by setting

$$y = r + \mathbb{E}_{\epsilon}[Q_{\theta'}(s', \pi_{\phi'}(s') + \epsilon)]$$

- ▶ So, the actual target is

$$y = r + [Q_{\theta'}(s', \pi_{\phi'}(s') + \epsilon)], \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \sigma, -c, c))$$

TD3 Algorithm

Algorithm 1 TD3

Initialize critic networks $Q_{\theta_1}, Q_{\theta_2}$, and actor network π_ϕ with random parameters θ_1, θ_2, ϕ

Initialize target networks $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$

Initialize replay buffer $\mathcal{B}$

for $t = 1$ **to** T **do**

 Select action with exploration noise $a \sim \pi_\phi(s) + \epsilon$,

$\epsilon \sim \mathcal{N}(0, \sigma)$ and observe reward r and new state s'

 Store transition tuple (s, a, r, s') in $\mathcal{B}$

 Sample mini-batch of N transitions (s, a, r, s') from $\mathcal{B}$

$\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c)$

$y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \tilde{a})$

 Update critics $\theta_i \leftarrow \text{argmin}_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$

if $t \bmod d$ **then**

 Update ϕ by the deterministic policy gradient:

$\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$

 Update target networks:

$\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$

$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$

end if

end for

Algorithm 1 TD3

Initialize critic networks $Q_{\theta_1}, Q_{\theta_2}$, and actor network π_ϕ with random parameters θ_1, θ_2, ϕ

Initialize target networks $\theta'_1 \leftarrow \theta_1, \theta'_2 \leftarrow \theta_2, \phi' \leftarrow \phi$

Initialize replay buffer $\mathcal{B}$

for $t = 1$ **to** T **do**

 Select action with exploration noise $a \sim \pi_\phi(s) + \epsilon$,

$\epsilon \sim \mathcal{N}(0, \sigma)$ and observe reward r and new state s'

 Store transition tuple (s, a, r, s') in $\mathcal{B}$

 Sample mini-batch of N transitions (s, a, r, s') from $\mathcal{B}$

$\tilde{a} \leftarrow \pi_{\phi'}(s') + \epsilon, \quad \epsilon \sim \text{clip}(\mathcal{N}(0, \tilde{\sigma}), -c, c)$

$y \leftarrow r + \gamma \min_{i=1,2} Q_{\theta'_i}(s', \tilde{a})$

 Update critics $\theta_i \leftarrow \text{argmin}_{\theta_i} N^{-1} \sum (y - Q_{\theta_i}(s, a))^2$

if $t \bmod d$ **then**

 Update ϕ by the deterministic policy gradient:

$\nabla_\phi J(\phi) = N^{-1} \sum \nabla_a Q_{\theta_1}(s, a)|_{a=\pi_\phi(s)} \nabla_\phi \pi_\phi(s)$

 Update target networks:

$\theta'_i \leftarrow \tau \theta_i + (1 - \tau) \theta'_i$

$\phi' \leftarrow \tau \phi + (1 - \tau) \phi'$

end if

end for
